

# 2026-07-27 更新:Gate v3 方法说明与跨数据集验证

## 一、背景

多跳问答系统每一跳检索候选证据后，用一个轻量分类器 ("gate") 判断该跳证据选择是否可能有误，判断结果以连续分数的形式与余弦相似度加权混合，参与候选排序：

$$\text{final\_score} = \text{emb\_score} - \lambda \cdot \text{gate\_score}$$

**emb\_score** 为候选与子问题的余弦相似度，**gate\_score** 为 gate 预测的"该跳证据选择有误"概率， $\lambda$  为固定权重（默认 0.25）。gate 的训练数据来自 MuSiQue（唯一提供逐跳 ground-truth 证据标注的数据集），推理阶段则可应用于 MuSiQue、2WikiMultihopQA、HotpotQA 三个数据集。

**gate v2**：对第  $j$  跳，取证据选定前后的前缀隐藏状态  $h_{\{j-1\}}$ 、 $h_j$ ，定义  $\Delta_j = h_j - h_{\{j-1\}}$ ，训练 **PCA(64) + Logistic Regression** 判断该跳是否选错。按跳位置分层： $j=0,1$  用 Llama 第 15 层， $j=2,3$  用第 23 层。

## 二、gate v3:特征层面融合

gate v2 只使用  $\Delta_j$ （局部变化信号）。另有一个已独立验证的信号——**h\_after**（证据选定后的绝对隐藏状态，代表"截至当前这一跳为止，整条推理前缀的状态"，累计/全局性质）。gate v3 将两者在特征层面融合，而非事后组合两个独立模型的输出分数：**h\_after** 与  $\Delta_j$  各自独立做 PCA（两者协方差结构不同，不共用一个 PCA），降维后的两组特征拼接，训练一个 Logistic Regression：

$$p_j = \sigma(w_h^T \text{PCA}(h_{\text{after}}) + w_{\Delta}^T \text{PCA}(\Delta_j) + b)$$

层的选择直接复用 gate v2 已验证的分层结果（ $j=0,1 \rightarrow$  第 15 层， $j=2,3 \rightarrow$  第 23 层），训练数据复用已提取的隐藏状态缓存，无需新的提取工作。推理阶段的组合公式与 gate v2 完全一致（**final\_score = emb\_score -  $\lambda$  · gate\_v3\_score**， $\lambda=0.25$ ），仅将 **gate\_score** 的计算方式替换为上述拼接特征打出的分数——因此 gate v3 与 gate v2 的对比是纯粹的"信号替换"，排序机制本身没有变化。

## 三、与 gate v2 的对比

### 3.1 分类指标（判断"该跳证据选择是否有误"的二分类任务，不涉及候选池排序）

数据源为 MuSiQue 固定的 gold/counterfactual trace：

|                                              | AUC    | 整体 F1  | 整体 TPR |
|----------------------------------------------|--------|--------|--------|
| gate v2 ( $\Delta$ only)                     | 0.9221 | 0.7785 | 0.8456 |
| gate v3 ( $h_{\text{after}} \oplus \Delta$ ) | 0.9357 | 0.793  | 0.8838 |

按跳位置拆分， $j=2$ （gate v2 原本最弱的一跳）F1 提升幅度最大（0.7535  $\rightarrow$  0.7919）。

### 3.2 端到端评测:MuSiQue

两者均使用相同的连续加权混合公式 ( $\lambda=0.25$ )，机制完全一致，是干净的同机制对比：

| 方法                                   | recall@1 | chain  | EM     | F1     |
|--------------------------------------|----------|--------|--------|--------|
| gate v2 ( $\Delta$ only)             | 0.6627   | 0.4357 | 0.4100 | 0.5045 |
| gate v3 ( $h\_after \oplus \Delta$ ) | 0.6959   | 0.5027 | 0.4245 | 0.5164 |

四项指标全面提升，chain（整条链路全对）提升幅度最大（+6.70 个百分点）。

### 3.3 端到端评测:2WikiMultihopQA、HotpotQA

gate v3 同样使用连续加权混合公式在两个数据集上评测：

| 数据集             | 方法      | recall@1 | chain  | EM     | F1     |
|-----------------|---------|----------|--------|--------|--------|
| 2WikiMultihopQA | gate v3 | 0.9001   | 0.7951 | 0.4808 | 0.5538 |
| HotpotQA        | gate v3 | 0.6425   | 0.5207 | 0.5095 | 0.6355 |

仓库中这两个数据集此前只运行过 gate v2 的二元阈值触发版本 ([gated\\_rule\\_a](#))，作为参考：

| 数据集             | 方法                                                | recall@1 | chain  | EM     | F1     |
|-----------------|---------------------------------------------------|----------|--------|--------|--------|
| 2WikiMultihopQA | gate v2 ( <a href="#">gated_rule_a</a> ，参考，非同一机制) | 0.8569   | 0.6819 | 0.4758 | 0.5499 |
| HotpotQA        | gate v2 ( <a href="#">gated_rule_a</a> ，参考，非同一机制) | 0.6257   | 0.4112 | 0.5129 | 0.6397 |

2WikiMultihopQA 上 gate v3 四项指标均高于参考基线；HotpotQA 上 gate v3 检索侧指标（recall@1、chain）明显高于参考基线，chain 提升尤为显著（+10.95 个百分点），但答案侧指标（EM、F1）略低于参考基线（EM -0.34、F1 -0.42 个百分点）。这两个数据集尚无 gate v2 连续加权 混合版本的历史结果，因此这组对比的参考基线与 gate v3 使用的机制不完全一致，仅作方向性参考，严格的同机制对比仅 MuSiQue 一项成立。

## 四、结论

gate v3 在 MuSiQue 上与 gate v2 的同机制对比中四项指标全面提升，证实特征层面融合 ( $h\_after$  与  $\Delta$  拼接后训练单一模型) 优于仅使用  $\Delta$  的原方案。在 2WikiMultihopQA 与 HotpotQA 上，gate v3 的检索侧指标同样明显优于现有参考基线；HotpotQA 的答案侧指标出现小幅下降，检索侧提升未能完全 转化为答案侧提升，可能与最终阅读理解模块本身的能力上限有关。

## 五、Selection 步骤的取舍与子问题占位符的训练/推理不一致

生产链路 ([retrieval/run\\_retrieval\\_exp\\_wavefront.py](#)) 每一跳的完整流程是：候选检索 → gate 打分 排序取 top-3 → 再调一次 LLM，让它从 top-3 里选一个 ([prompt\\_select\\_passage\\_with\\_context](#)) → 用选中的证据生成这一跳的短答案 → 短答案用于展开下一跳子问题里的 [\[Answer N\]](#) 占位符。这一版本 记录两套指标：排序阶段的 [recall@1](#) (gold 是不是排第一) 和 LLM 选择之后的 [selection\\_acc](#) (LLM 最终选的是不是 gold)。

gate v3 (retrieval/run\_retrieval\_exp\_wavefront\_gate\_v3.py) 和更早的 combined\_gate\_rerank 则没有这个 LLM selection 步骤，排序完直接取 rank-1 (或 top-beam\_width) 作为最终证据。

5.1 selection 步骤：验证后确认应该去掉

用生产脚本 gated\_rule\_a (gate\_v2, musique, 20260706\_153235\_gate\_v2\_single\_gpu 那次跑的数据) 对比排序阶段的 recall@1 和 LLM selection 之后的 selection\_acc：

|     | recall@1 (排序阶段) | selection_acc (LLM 选完之后) |
|-----|-----------------|--------------------------|
| 整体  | 0.6565          | 0.6185                   |
| K=2 | 0.7188          | 0.6736                   |
| K=3 | 0.6212          | 0.5906                   |
| K=4 | 0.5526          | 0.5179                   |

每一个 K 桶，LLM selection 之后的准确率都比排序阶段的 rank-1 准确率低——说明在这套流程里，让 LLM 在候选里再选一次，反而比直接相信排序结果更差。基于这个发现，gate v3 (retrieval/run\_retrieval\_exp\_wavefront\_gate\_v3.py) 和更早的 combined\_gate\_rerank 都去掉了 这一步，排序完直接取 rank-1 (或 top-beam\_width) 作为最终证据，不再让 LLM 二次选择。

5.2 由此发现的一个真实的训练/推理不一致

MuSiQue 的子问题 (GT decompose 生成) 里，后续跳经常需要引用前面跳的答案，写法是 [Answer N] 这样的占位符 (例如 "When was [Answer 1] founded?")。统计真实数据 (3000 条 train 样本，6000 跳)：

总跳数：6000  
子问题里带 [Answer N] 占位符 (未展开) 的跳数：2940 (49.0%)

gate v2/v3 训练时读取的 hidden\_states/pilot\_multilayer，来自 traces/merged/musique/\*.jsonl 的 reasoning\_trace 字段——这个字段里子问题的占位符是没有展开的 (assemble\_trace() 存进去的 就是原始子问题文本，expand\_hop\_template() 从未在这条链路上被调用过)。也就是说，gate v2/v3 训练时，约一半的跳，看到的子问题字面上就带着 [Answer N]。

但 run\_retrieval\_exp\_wavefront\_gate\_v3.py 推理时，子问题是展开过的：

```
expanded_q = expand_hop_template(raw_sq, path.prior)           # 用逐跳短答案展开占位符
candidates_all = embed_retrieval(expanded_q, ...)             # 展开后的文本用于检索
text = f'{prefix_before} Step {hop_j}: {expanded_q} Evidence: "..."'  # 展开后的文本喂给 gate 打分
```

这是一个此前没有被注意到的训练/推理不一致：gate v3 训练时看到"未展开"的子问题文本，推理时 却被喂"已展开"的子问题文本去打分，而且影响的正好是约一半的跳位置。

### 5.3 提出的方案（尚未实测，是下一步要做的验证）

去掉逐跳短答案生成这一环节，子问题**始终使用原始（未展开）文本**，用于检索和喂给 gate 打分两处：

- 检索：直接用原始子问题去检索，不再展开 [Answer N]
- gate 打分：喂给 gate 的文本也用原始子问题——跟训练时的分布完全一致，消除上面发现的不一致
- 不再生成逐跳短答案，每一跳只保留"原始子问题 + 选中的 top-1 证据"这一对
- 所有跳结束后，把每一跳的"原始子问题 + 选中证据"一次性全部交给最终 reader，由它一次性推理出最终答案（不再看任何中间生成的短答案）

预期能同时带来三点好处：

1. 消除上面发现的 gate 训练/推理不一致
2. 每条样本少生成 K 次逐跳短答案，减少一次 LLM 调用
3. 消除之前发现的"某跳短答案生成失败输出 NA、污染后续所有跳"的级联失败模式（musique dev 全量里约 9.2% 的样例最终卡在这个问题上）

**唯一需要实跑验证、不能假设的风险：**原始子问题（带字面占位符）用于**检索**这一步，语义上是 残缺的，可能会拖累候选段落检索到的质量，尤其是那约 49% 需要靠前面跳答案才能问清楚的"桥接"子 问题。这个风险集中在\*\*余弦检索（BGE）\*\*这一步——BGE 是通用语义模型，从未在这条多跳链路上训练 过，原始子问题剩下的词不够有区分度，完全是经验问题，没法靠推理判断。**gate v3 自身的判断不会 因此受影响**（它本来就是在这种未展开文本上训练的，现在推理时用同样风格的文本，只会更一致、不会 更差）——风险只在候选池这一步，不在打分这一步。

**当前状态：**方案已确定，还没有写代码、没有实跑数据——这是下一步要做的验证。